

姓名： 陈英杰

学号： 20232673

《GNSS 定位新技术及数据处理方法》课后大作业（二）

任务二：导航定位手机 APP 改进设计与实现

任务部分：基于 Android 或 iOS 平台或鸿蒙平台，在已有代码基础上改进导航定位手机 APP，实现 2 项新功能，例如电子围栏、拍照定位、基于 CORS 或 PPP 提升定位精度等。

本次作业以 Android 平台为实现对象，在已有高德地图定位 APP 基础上，重点完成“电子围栏”和“拍照定位”两项新功能，同时进一步补充了地点搜索、步行/驾车路线规划、轨迹记录以及界面交互优化，形成了一个可直接演示的导航定位应用原型。

1 算法

1.1 任务理解与系统总体目标

本次任务的核心不是重新从零开发地图应用，而是在已有导航定位程序基础上完成功能增强与工程化完善。因此，系统设计需要同时满足三方面要求：第一，能够稳定调用地图 SDK 完成实时定位与地图显示；第二，新增至少两项与导航定位业务紧密相关的实用功能；第三，界面与交互要便于课堂演示和结果展示。结合当前工程代码，本文将“电子围栏”和“拍照定位”作为两项核心新功能，并将“地点搜索—目的地选择—路线规划—轨迹记录”组织为辅助增强链路，从而使整个 APP 的演示逻辑更加完整。

从软件结构上看，本系统采用“界面层 + 地图服务层 + 业务逻辑层”的组织方式。activity_main.xml 负责界面布局，MainActivity.java 统一管理地图对象、定位状态、事件监听与功能逻辑，AndroidManifest.xml 完成权限声明与高德地图 Key 注册。整个应用遵循 Android Activity 生命周期，在 onCreate 中初始化地图与控件，在 onResume/onPause/onDestroy 中与 MapView 生命周期保持一致，从而保证地图组件与定位资源能够正确创建、暂停和释放。

1.2 系统总体流程

程序启动后，系统首先完成 SDK 隐私合规初始化、界面控件绑定、地图对象创建以及路线搜索对象 `RouteSearch` 的实例化；随后请求定位权限并开启高精度定位。定位成功后，程序实时刷新当前位置、地址信息和定位精度，并在首次定位时自动将地图镜头移动到当前点。用户在此基础上可继续执行地点搜索、路线规划、电子围栏设置、拍照定位和轨迹记录等操作。

从交互流角度看，本系统可概括为以下逻辑链：① 启动并定位；② 若用户需要导航，则先搜索地点并从结果列表中选中目的地，再规划步行或驾车路线；③ 若用户需要区域监测，则开启围栏模式并长按地图设置围栏中心，再由程序持续判断当前位置是否进入或离开围栏；④ 若用户需要现场记录，则直接拍照并把图像与当前时间、地址、经纬度绑定为一个拍照定位点；⑤ 若用户需要移动过程回放，则开启轨迹记录并在运动过程中持续绘制折线。

1.3 定位与地图显示机制

定位部分由高德地图定位 SDK 完成。代码中通过 `AMapLocationClientOption` 将定位模式设置为 `Hight_Accuracy`，并开启持续定位、地址逆解析和 2 s 更新间隔。该设置意味着系统优先输出适合移动端地图应用的实时位置解，不仅返回经纬度，还返回当前地址和精度信息，便于界面展示与后续业务逻辑复用。

$$L_t = (\varphi_t, \lambda_t, a_t)$$

其中， L_t 表示时刻 t 的位置状态， φ_t 与 λ_t 分别为纬度和经度， a_t 为定位精度估计值。程序在 `onLocationChanged` 回调中更新当前位置 `currentLatLng`、当前地址 `currentAddress` 和当前城市 `currentCity`，并将其同步显示到顶部状态卡片中。首次定位成功后，地图通过 `animateCamera` 自动聚焦到当前点，提高了进入应用后的可用性。

1.4 电子围栏算法

电子围栏功能的本质是一个“点—圆域”空间判定问题。用户先开启围栏模式，再通过长按地图给出围栏中心 $C=(\varphi_0, \lambda_0)$ ，同时通过滑动条设置半径 r 。之后系统对每一个新的定位点 L_t 计算其到围栏中心的球面距离 d_t ，并据此判断用户处于围栏内还是围栏外。

$$d_t = 2R_e \cdot \arcsin \sqrt{(\sin^2((\varphi_t - \varphi_0)/2) + \cos \varphi_t \cos \varphi_0 \sin^2((\lambda_t - \lambda_0)/2))}$$
$$state_t = \{ inside, d_t \leq r; outside, d_t > r \}$$

在工程实现中，距离计算调用 `AMapUtils.calculateLineDistance` 完成，避免了手工处理地球曲率与投影细节。程序使用变量 `lastInsideState` 记录上一次围栏判定结果，

仅在状态发生变化时弹出“已进入电子围栏”或“已离开电子围栏”的提示，从而避免持续定位条件下重复弹窗。该策略本质上属于状态机去抖处理，可显著改善交互体验。

围栏半径由 SeekBar 动态控制。代码将进度条 progress 与实际半径通过 $r = \text{progress} + 50$ 建立对应关系，使得半径可在 50 m 到 500 m 之间连续变化。每次半径变化后，程序都会同步更新地图圆形覆盖物的半径，并立刻重新检查当前位置相对于围栏的状态，因此界面显示与判定结果具有一致性。

1.5 拍照定位算法

拍照定位功能的目的是把现场图像信息与空间位置信息绑定起来，形成一个可回看、可点击的地理标记点。程序在启动相机前先缓存当前经纬度、地址和时间信息，待相机返回后，再将拍摄图像与这些元数据共同写入内存对象。其本质可以表示为如下数据结构：

$$P_i = \{I_i, \varphi_i, \lambda_i, t_i, A_i\}$$

其中， I_i 为拍摄图像， (φ_i, λ_i) 为拍摄位置， t_i 为拍摄时间， A_i 为逆地理编码得到的地址字符串。程序在得到相机返回结果后，会在地图上添加一个玫红色 Marker，并以 Marker 的唯一 ID 作为键，将图像和文字说明分别存入 photoBitmapMap 与 photoInfoMap。随后，当用户点击该拍照点时，系统根据 marker.getId() 反查出对应图像与文字信息，并通过对话框展示缩略图、拍摄时间、地址以及经纬度。

该方案的优点是实现简单、演示效果直观，特别适合课程设计或原型验证场景。需要说明的是，当前实现通过 MediaStore.ACTION_IMAGE_CAPTURE 获取的是相机返回的缩略图 Bitmap，并未将原始大图文件持久化存储到外部路径。因此它更偏向“拍照定位原型验证”，若后续面向实际业务部署，可进一步扩展为：保存原图、写入 EXIF 坐标、建立本地数据库索引以及支持历史照片检索。

1.6 轨迹记录算法

轨迹记录用于描述用户在移动过程中的路径变化。若每次定位更新都无差别写入轨迹，会引入大量几乎重合的点，导致折线抖动明显、轨迹距离累积偏大，也不利于界面渲染效率。因此本程序采用了一个最小距离阈值采样策略：仅当新位置与上一轨迹点之间的距离不小于 2 m 时，才将该点纳入轨迹。

$$\begin{aligned} \Delta s_k &= D(p_k, p_{(k-1)}) \\ p_k \in \text{Track} &\Leftrightarrow \Delta s_k \geq d_{\min}, d_{\min} = 2\text{ m} \\ S_n &= \sum \Delta s_k \end{aligned}$$

其中, p_k 为第 k 个候选位置点, Δs_k 为其与上一已保存轨迹点之间的距离, S_n 为累计轨迹长度。程序使用 `ArrayList<LatLng>` 存储轨迹点, 并利用 `Polyline` 在地图上实时绘制轨迹线; 同时以变量 `trackDistance` 累加总路径长度, 并在顶部状态栏显示“点数 + 距离”的实时结果。该处理方式在保证轨迹连贯性的同时, 抑制了静止状态下的微小定位噪声。

1.7 地点搜索与路径规划算法

虽然本次作业重点是新增功能, 但从实际演示效果来看, 单纯展示电子围栏或拍照定位还不够完整, 因此本程序进一步集成了地点搜索与路径规划功能。地点搜索通过 `PoiSearch.Query(keyword, "", "")` 构造关键字查询, 随后异步返回一组 POI 结果; 用户点击结果列表中的某一项后, 程序提取其 `LatLonPoint` 坐标作为 `routeDestination`, 并在地图上放置橙色目的地 `Marker`。

路线规划部分分别调用步行和驾车两种查询接口。设当前点为 P_s , 目的地为 P_e , 则路径搜索对象可记为:

$$\Gamma = \{p_0 = P_s, p_1, p_2, \dots, p_n = P_e\}$$

路径求解完成后, 程序把每个 `Step` 中的 `polyline` 点序列提取出来, 拼接为完整折线并绘制到地图上。步行路线使用蓝色折线, 驾车路线使用橙色折线, 从而在视觉上便于区分。与此同时, 系统还读取 `SDK` 返回的路径长度与预计耗时, 并在状态栏中显示“步行/驾车 + 距离 + 约用时”的摘要信息。

路线绘制完成后, 程序通过遍历所有路径点构建 `LatLngBounds`, 再调用 `animateCamera(newLatLngBounds(...))` 自动调整视野, 使整条路线完整显示在屏幕中。这一处理避免了只显示起点或终点的局部视图, 有利于结果演示。

1.8 权限控制与生命周期管理

移动端定位类应用必须正确处理权限与生命周期, 否则容易出现定位失败、相机调用异常或地图资源泄漏等问题。当前工程在 `AndroidManifest.xml` 中声明了网络、粗/精定位、Wi-Fi 状态、唤醒锁和相机等权限; 在运行时, 则分别对定位权限和相机权限进行检查与申请。若用户拒绝授权, 程序通过 `Toast` 给出明确提示, 避免因静默失败造成使用困惑。

在生命周期方面, `MainActivity` 中显式实现了 `onResume`、`onPause`、`onSaveInstanceState` 和 `onDestroy`, 并调用 `MapView` 对应方法, 使地图组件与 `Activity`

生命周期保持同步。定位客户端 `aMapLocationClient` 在 `deactivate` 和 `onDestroy` 中被停止并销毁，防止后台持续占用定位资源。该部分虽不直接体现算法复杂性，但对于保证程序长期运行的稳定性非常关键。

2 代码

2.1 工程文件组成与作用说明

当前对话中上传的工程文件包括 4 个主要代码/配置文件：顶层 `build.gradle`、`AndroidManifest.xml`、`activity_main.xml` 和 `MainActivity.java`。由于未提供 module 级 `build.gradle`、`strings.xml`、`colors.xml` 等资源文件，以下“完整代码”部分按照当前已上传内容原样附录，已能够覆盖本次作业展示所需的核心实现逻辑。

表 2-1 工程文件说明

文件名	类型	主要作用
build.gradle	Gradle 顶层配置	定义项目级插件入口，为 Android 工程构建提供 总计配置框架。
AndroidManifest.xml	应用清单文件	声明网络、定位、相机等权限，注册高德地图 Key，并声明主 Activity。
activity_main.xml	界面布局文件	定义 MapView 、搜索栏、状态卡片、围栏半 径卡片、底部功能区和定位按钮等界面元素。
MainActivity.java	核心逻辑文件	实现地图初始化、实时定位、电子围栏、拍照 定位、轨迹记录、 POI 搜索、路线规划、权限 处理和交互逻辑。

2.2 关键代码设计说明

(1) **AndroidManifest.xml** 负责应用运行的基础环境声明。从代码可见，程序显式申请了 **INTERNET**、**ACCESS_FINE_LOCATION**、**ACCESS_COARSE_LOCATION**、**CAMERA** 等权限，并通过 **<meta-data>** 节点写入高德地图 **Key**。这保证了地图渲染、网络搜索、实时定位与拍照功能的前提条件。

(2) `activity_main.xml` 采用“地图全屏 + 顶部信息卡片 + 底部功能卡片 + 右侧定位按钮”的布局思路。顶部卡片负责搜索与状态展示，底部卡片采用两行窄按钮实现围栏、拍照和轨迹等功能入口，避免界面显得臃肿；围栏半径则通过单独的小卡片展示，使参数调节与功能按钮分离，交互更清晰。

（3）MainActivity.java 是整个应用的控制中枢。该文件同时实现了 LocationSource 、 AMapLocationListener 、 OnMapLongClickListener 、 OnMarkerClickListener 、 PoiSearch.OnPoiSearchListener 和

RouteSearch.OnRouteSearchListener 等多个接口，从而在一个 Activity 内统一接收定位更新、长按设围栏、点击拍照点、POI 搜索结果和路线规划结果等事件。

(4) 围栏、轨迹与拍照功能的数据结构设计较为清晰：围栏中心使用 LatLng fenceCenter 保存，围栏半径由 fenceRadius 记录；轨迹点用 ArrayList<LatLng> trackPoints 保存并用 trackDistance 累加长度；拍照定位使用两个 HashMap 以 Marker ID 为键分别保存 Bitmap 和文字说明。这种“地图对象 + 业务数据字典”的组织方式有利于后续扩展，例如未来可把 HashMap 替换为数据库，实现历史记录持久化。

2.3 完整代码附录

以下代码为当前已上传工程文件的完整内容，按照配置文件、清单文件、布局文件 and 核心逻辑文件的顺序依次给出，便于教师检查与课程报告留档。

2.3.1 build.gradle (顶层)

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace 'com.example.gpsmap'
    compileSdk {
        version = release(36) {
            minorApiLevel = 1
        }
    }

    defaultConfig {
        applicationId "com.example.gpsmap"
        minSdk 24
        targetSdk 36
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
        ndk {
            abiFilters 'armeabi-v7a', 'arm64-v8a'
        }
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_11
        targetCompatibility JavaVersion.VERSION_11
    }
}

dependencies {
    implementation libs.appcompat
    implementation libs.material
    testImplementation libs.junit
    androidTestImplementation libs.ext.junit
    androidTestImplementation libs.espresso.core
    implementation 'com.amap.api:3dmap-location-search:latest.integration'
```

2.3.2 AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
    <uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.CHANGE_WIFI_STATE" />
    <uses-permission android:name="android.permission.WAKE_LOCK" />
    <uses-permission android:name="android.permission.CAMERA" />

    <uses-feature
        android:name="android.hardware.camera"
        android:required="false" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.GPSmap"
        tools:targetApi="31">

        <meta-data
            android:name="com.amap.api.v2.apikey"
            android:value="86c47ce5ealed02ff40457e316d0c6a8" />

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>

</manifest>
```

2.3.3 activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
```

主界面布局说明:

1. 最底层是高德地图 MapView
2. 顶部是搜索与状态卡片，支持“展开/收起”
3. 右侧是单独的“回到定位”圆形按钮
4. 底部是两行窄版功能按钮区
5. 围栏半径单独使用一个小卡片显示，避免和按钮混在一起显得拥挤

-->

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
<!-- 地图主体 -->
```

```
<com.amap.api.maps.MapView
    android:id="@+id/mapView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

```
<!-- 顶部搜索与状态卡片 -->
```

```
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginStart="12dp"
    android:layout_marginTop="12dp"
    android:layout_marginEnd="12dp"
    app:cardBackgroundColor="#F7FFFFFF"
    app:cardCornerRadius="18dp"
    app:cardElevation="5dp"
    app:strokeColor="#E2E8F0"
    app:strokeWidth="1dp">
```

<LinearLayout

```
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical"
android:padding="12dp">
```

<!-- 顶部标题 + 展开收起 -->

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center_vertical"
    android:orientation="horizontal">
```

<TextView

```
android:layout_width="0dp"
android:layout_height="wrap_content"
android:layout_weight="1"
android:text="GPSmap"
android:textColor="#1F2937"
android:textSize="20sp"
android:textStyle="bold" />
```

```
<com.google.android.material.button.MaterialButton
```

```

android:id="@+id/btnToggleInfo"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:minHeight="0dp"
android:minWidth="0dp"
android:paddingStart="10dp"
android:paddingTop="4dp"
android:paddingEnd="10dp"
android:paddingBottom="4dp"
android:text="收起"
android:textAllCaps="false"
android:textColor="#FFFFFF"
android:textSize="11sp"
app:backgroundTint="#64748B"
app:cornerRadius="14dp" />

```

</LinearLayout>

<!-- 搜索栏 -->

```
<LinearLayout
```



```

        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:orientation="horizontal">

        <com.google.android.material.card.MaterialCardView
            android:layout_width="0dp"
            android:layout_height="40dp"
            android:layout_weight="1"
            app:cardBackgroundColor="#F8FAFC"
            app:cardCornerRadius="14dp"
            app:cardElevation="0dp"
            app:strokeColor="#CBD5E1"
            app:strokeWidth="1dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:gravity="center_vertical"
                android:orientation="horizontal"
                android:paddingStart="10dp"
                android:paddingEnd="10dp">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="0"
                    android:textColor="#64748B"
                    android:textSize="14sp" />

                <EditText
                    android:id="@+id/etKeyword"
                    android:layout_width="0dp"
                    android:layout_height="match_parent"
                    android:layout_marginStart="6dp"
                    android:layout_weight="1"
                    android:background="@android:color/transparent"
                    android:hint="搜索位置"
                    android:textColor="#0F172A"
                    android:textColorHint="#94A3B8"
                    android:textSize="13sp" />

            </LinearLayout>
        </com.google.android.material.card.MaterialCardView>

        <com.google.android.material.button.MaterialButton
            android:id="@+id/btnSearch"
            android:layout_width="wrap_content"
            android:layout_height="40dp"
            android:layout_marginStart="8dp"
            android:minWidth="0dp"
            android:text="搜索"
            android:textAllCaps="false"
            android:textColor="#FFFFFF"
            android:textSize="12sp"
            app:backgroundTint="#475569"
            app:cornerRadius="14dp" />
    </LinearLayout>

    <!-- 这一块支持展开 / 收起 -->
    <LinearLayout
        android:id="@+id/layoutSearchExtras"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- 搜索结果列表 -->
        <ListView
            android:id="@+id/lvSearchResults"
            android:layout_width="match_parent"
            android:layout_height="190dp"
            android:layout_marginTop="8dp"
            android:divider="@android:color/transparent"
            android:dividerHeight="6dp"
            android:padding="2dp"
            android:clipToPadding="false"
            android:visibility="gone" />

        <!-- 路线按钮，只有选中目的地才出现 -->
        <LinearLayout
            android:id="@+id/layoutRouteActions"

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:orientation="vertical"
        android:visibility="gone">

<TextView
    android:id="@+id/tvSelectedDestination"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="已选目的地: --"
    android:textColor="#0F172A"
    android:textSize="12sp"
    android:textStyle="bold" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:orientation="horizontal">

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnWalkRoute"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="步行"
        android:textAllCaps="false"
        android:textColor="#FFFFFF"
        android:textSize="11sp"
        app:backgroundTint="#64748B"
        app:cornerRadius="14dp" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnDriveRoute"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="驾车"
        android:textAllCaps="false"
        android:textColor="#FFFFFF"
        android:textSize="11sp"
        app:backgroundTint="#78716C"
        app:cornerRadius="14dp" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnClearRoute"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="清路"
        android:textAllCaps="false"
        android:textColor="#FFFFFF"
        android:textSize="11sp"
        app:backgroundTint="#94A3B8"
        app:cornerRadius="14dp" />

</LinearLayout>
</LinearLayout>

<!-- 状态文本 -->
<TextView
    android:id="@+id/tvAddress"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:maxLines="2"
    android:text="地址: 定位中..."
    android:textColor="#475569"
    android:textSize="12sp" />

<TextView
    android:id="@+id/tvCoord"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"

```

```

        android:layout_marginTop="3dp"
        android:text="经纬度: --"
        android:textColor="#475569"
        android:textSize="12sp" />

<TextView
    android:id="@+id/tvFenceStatus"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="3dp"
    android:text="围栏: 未设置"
    android:textColor="#64748B"
    android:textSize="12sp"
    android:textStyle="bold" />

<TextView
    android:id="@+id/tvTrackStatus"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="3dp"
    android:text="轨迹: 未记录"
    android:textColor="#64748B"
    android:textSize="12sp"
    android:textStyle="bold" />

<TextView
    android:id="@+id/tvRouteStatus"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="3dp"
    android:text="路线: 未规划"
    android:textColor="#64748B"
    android:textSize="12sp"
    android:textStyle="bold" />
</LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- 围栏半径小卡片 -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/cardFenceRadius"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    android:layout_marginStart="12dp"
    android:layout_marginEnd="12dp"
    android:layout_marginBottom="86dp"
    android:visibility="gone"
    app:cardBackgroundColor="#F7FFFFFF"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp"
    app:strokeColor="#E2E8F0"
    app:strokeWidth="1dp">

    <LinearLayout
        android:id="@+id/layoutFenceRadius"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="10dp">

        <TextView
            android:id="@+id/tvFenceRadius"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="围栏半径: 100m"
            android:textColor="#334155"
            android:textSize="12sp"
            android:textStyle="bold" />

        <SeekBar
            android:id="@+id/seekFenceRadius"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- 底部功能条 -->

```

```

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    android:layout_marginStart="12dp"
    android:layout_marginEnd="12dp"
    android:layout_marginBottom="12dp"
    app:cardBackgroundColor="#F7FFFFFF"
    app:cardCornerRadius="18dp"
    app:cardElevation="5dp"
    app:strokeColor="#E2E8F0"
    app:strokeWidth="1dp">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="8dp">

<!-- 第一行 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnFenceMode"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="围栏"
        android:textAllCaps="false"
        android:textColor="#334155"
        android:textSize="10sp"
        app:backgroundTint="#E2E8F0"
        app:cornerRadius="14dp" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnTakePhoto"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="拍照"
        android:textAllCaps="false"
        android:textColor="#334155"
        android:textSize="10sp"
        app:backgroundTint="#E2E8F0"
        app:cornerRadius="14dp" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnTrack"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="轨迹"
        android:textAllCaps="false"
        android:textColor="#334155"
        android:textSize="10sp"
        app:backgroundTint="#E2E8F0"
        app:cornerRadius="14dp" />
</LinearLayout>

<!-- 第二行 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:orientation="horizontal">

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnClearFence"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_marginEnd="6dp"

```

```

        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="清圈"
        android:textAllCaps="false"
        android:textColor="#334155"
        android:textSize="10sp"
        app:backgroundTint="#E2E8F0"
        app:cornerRadius="14dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/btnClearPhotos"
    android:layout_width="0dp"
    android:layout_height="34dp"
    android:layout_marginEnd="6dp"
    android:layout_weight="1"
    android:minWidth="0dp"
    android:text="清拍"
    android:textAllCaps="false"
    android:textColor="#334155"
    android:textSize="10sp"
    app:backgroundTint="#E2E8F0"
    app:cornerRadius="14dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/btnClearTrack"
    android:layout_width="0dp"
    android:layout_height="34dp"
    android:layout_weight="1"
    android:minWidth="0dp"
    android:text="清轨"
    android:textAllCaps="false"
    android:textColor="#334155"
    android:textSize="10sp"
    app:backgroundTint="#E2E8F0"
    app:cornerRadius="14dp" />
</LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- 侧边回到定位按钮 -->
<com.google.android.material.button.MaterialButton
    android:id="@+id/btnCenterMe"
    android:layout_width="50dp"
    android:layout_height="50dp"
    android:layout_gravity="end|center_vertical"
    android:layout_marginEnd="14dp"
    android:minWidth="0dp"
    android:minHeight="0dp"
    android:text="⬅"
    android:textAllCaps="false"
    android:textColor="FFFFFF"
    android:textSize="18sp"
    app:backgroundTint="#475569"
    app:cornerRadius="25dp" />

</FrameLayout>

```

2.3.4 MainActivity.java

```
package com.example.gpsmap;

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.graphics.Bitmap;
import android.graphics.Typeface;
import android.graphics.drawable.GradientDrawable;
import android.os.Bundle;
import android.provider.MediaStore;
import android.util.TypedValue;
import android.view.Gravity;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.ListView;
import android.widget.SeekBar;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

import com.amap.api.location.AMapLocation;
import com.amap.api.location.AMapLocationClient;
import com.amap.api.location.AMapLocationClientOption;
import com.amap.api.location.AMapLocationListener;
import com.amap.api.maps.AMap;
import com.amap.api.maps.AMapUtils;
import com.amap.api.maps.CameraUpdateFactory;
import com.amap.api.maps.LocationSource;
import com.amap.api.maps.MapView;
import com.amap.api.maps.MapInitializer;
import com.amap.api.maps.model.BitmapDescriptorFactory;
import com.amap.api.maps.model.Circle;
import com.amap.api.maps.model.CircleOptions;
import com.amap.api.maps.model.LatLng;
import com.amap.api.maps.model.LatLngBounds;
import com.amap.api.maps.model.Marker;
import com.amap.api.maps.model.MarkerOptions;
import com.amap.api.maps.model.MyLocationStyle;
import com.amap.api.maps.model.Polyline;
import com.amap.api.maps.model.PolylineOptions;
import com.amap.api.services.core.AMapException;
import com.amap.api.services.core.LatLonPoint;
import com.amap.api.services.core.PoiItem;
import com.amap.api.services.poisearch.PoiResult;
import com.amap.api.services.poisearch.PoiSearch;
import com.amap.api.services.route.BusRouteResult;
import com.amap.api.services.route.DrivePath;
import com.amap.api.services.route.DriveRouteResult;
import com.amap.api.services.route.RideRouteResult;
import com.amap.api.services.route.RouteSearch;
import com.amap.api.services.route.WalkPath;
import com.amap.api.services.route.WalkRouteResult;
import com.amap.api.services.route.WalkStep;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Date;
import java.util.Locale;

public class MainActivity extends AppCompatActivity implements
    LocationSource,
    AMapLocationListener,
    AMap.OnMapLongClickListener,
    AMap.OnMarkerClickListener,
    PoiSearch.OnPoiSearchListener,
    RouteSearch.OnRouteSearchListener {
```

```

private static final int REQUEST_LOCATION_PERMISSION = 1001;
private static final int REQUEST_CAMERA_PERMISSION = 1002;
private static final int REQUEST_IMAGE_CAPTURE = 2001;

private MapView mapView;
private AMap aMap;

private EditText etKeyword;
private ListView lvSearchResults;

private TextView tvAddress;
private TextView tvCoord;
private TextView tvFenceStatus;
private TextView tvTrackStatus;
private TextView tvRouteStatus;
private TextView tvFenceRadius;
private TextView tvSelectedDestination;

private LinearLayout layoutSearchExtras;
private LinearLayout layoutRouteActions;
private LinearLayout layoutFenceRadius;
private View cardFenceRadius;

private Button btnToggleInfo;
private Button btnSearch;
private Button btnWalkRoute;
private Button btnDriveRoute;
private Button btnClearRoute;

private Button btnFenceMode;
private Button btnClearFence;

private Button btnTakePhoto;
private Button btnClearPhotos;

private Button btnTrack;
private Button btnClearTrack;

private Button btnCenterMe;

private SeekBar seekFenceRadius;

private boolean isInfoExpanded = true;

private OnLocationChangeListener mListener;
private AMapLocationClient aMapLocationClient;
private AMapLocationClientOption aMapLocationClientOption;

private boolean isFirstLocate = true;
private LatLng currentLatLng;
private String currentAddress = "未知地址";
private String currentCity = "";

// 围栏
private boolean isFenceModeEnabled = false;
private LatLng fenceCenter;
private Circle fenceCircle;
private Marker fenceMarker;
private float fenceRadius = 100f;
private Boolean lastInsideState = null;

// 轨迹
private boolean isTracking = false;
private final ArrayList<LatLng> trackPoints = new ArrayList<>();
private Polyline trackPolyline;
private float trackDistance = 0f;

// 搜索与路线
private RouteSearch routeSearch;
private Marker destinationMarker;
private LatLng routeDestination;
private String routeDestinationName = "";
private Polyline routePolyline;

```

```

private final ArrayList<PoiItem> searchPoiList = new ArrayList<>();
private SearchResultAdapter searchAdapter;

// 拍照点
private LatLng pendingPhotoLatLng;
private String pendingPhotoTime;
private String pendingPhotoAddress;
private final HashMap<String, Bitmap> photoBitmapMap = new HashMap<>();
private final HashMap<String, String> photoInfoMap = new HashMap<>();
private final ArrayList<Marker> photoMarkers = new ArrayList<>();
private final SimpleDateFormat timeFormat =
    new SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.getDefault());

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    if (getSupportActionBar() != null) {
        getSupportActionBar().hide();
    }

    MapsInitializer.updatePrivacyShow(this, true, true);
    MapsInitializer.updatePrivacyAgree(this, true);
    AMapLocationClient.updatePrivacyShow(this, true, true);
    AMapLocationClient.updatePrivacyAgree(this, true);

    setContentView(R.layout.activity_main);

    initView();

    mapView.onCreate(savedInstanceState);

    if (aMap == null) {
        aMap = mapView.getMap();
    }

    try {
        routeSearch = new RouteSearch(this);
        routeSearch.setRouteSearchListener(this);
    } catch (AMapException e) {
        e.printStackTrace();
    }

    initMap();
    initEvents();
    requestLocationPermissionIfNeeded();

    updateFenceStatusNoFence();
    updateTrackStatus();
    tvRouteStatus.setText("路线: 未规划");
}

private void initView() {
    mapView = findViewById(R.id.mapView);

    etKeyword = findViewById(R.id.etKeyword);
    lvSearchResults = findViewById(R.id.lvSearchResults);

    tvAddress = findViewById(R.id.tvAddress);
    tvCoord = findViewById(R.id.tvCoord);
    tvFenceStatus = findViewById(R.id.tvFenceStatus);
    tvTrackStatus = findViewById(R.id.tvTrackStatus);
    tvRouteStatus = findViewById(R.id.tvRouteStatus);
    tvFenceRadius = findViewById(R.id.tvFenceRadius);
    tvSelectedDestination = findViewById(R.id.tvSelectedDestination);

    layoutSearchExtras = findViewById(R.id.layoutSearchExtras);
    layoutRouteActions = findViewById(R.id.layoutRouteActions);
    layoutFenceRadius = findViewById(R.id.layoutFenceRadius);
    cardFenceRadius = findViewById(R.id.cardFenceRadius);

    btnToggleInfo = findViewById(R.id.btnToggleInfo);
    btnSearch = findViewById(R.id.btnSearch);
}

```



```

        btnWalkRoute = findViewById(R.id.btnWalkRoute);
        btnDriveRoute = findViewById(R.id.btnDriveRoute);
        btnClearRoute = findViewById(R.id.btnClearRoute);

        btnFenceMode = findViewById(R.id.btnFenceMode);
        btnClearFence = findViewById(R.id.btnClearFence);

        btnTakePhoto = findViewById(R.id.btnTakePhoto);
        btnClearPhotos = findViewById(R.id.btnClearPhotos);

        btnTrack = findViewById(R.id.btnTrack);
        btnClearTrack = findViewById(R.id.btnClearTrack);

        btnCenterMe = findViewById(R.id.btnCenterMe);

        seekFenceRadius = findViewById(R.id.seekFenceRadius);

        searchAdapter = new SearchResultAdapter();
        lvSearchResults.setAdapter(searchAdapter);
    }

    private void initMap() {
        aMap.getUiSettings().setScaleControlsEnabled(true);
        aMap.getUiSettings().setMyLocationButtonEnabled(false);
        aMap.getUiSettings().setZoomControlsEnabled(false);

        aMap.setOnMapLongClickListener(this);
        aMap.setOnMarkerClickListener(this);

        MyLocationStyle myLocationStyle = new MyLocationStyle();
        myLocationStyle.myLocationType(MyLocationStyle.LOCATION_TYPE_LOCATION_ROTATE_NO_CENTER);
        myLocationStyle.interval(2000);
        myLocationStyle.radiusFillColor(0x180EA5E9);
        myLocationStyle.strokeColor(0x440EA5E9);
        myLocationStyle.strokeWidth(1f);
        aMap.setMyLocationStyle(myLocationStyle);
    }

    private void initEvents() {
        seekFenceRadius.setMax(450);
        seekFenceRadius.setProgress(50);
        tvFenceRadius.setText("围栏半径: 100m");

        seekFenceRadius.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
            @Override
            public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
                fenceRadius = progress + 50;
                tvFenceRadius.setText("围栏半径: " + (int) fenceRadius + "m");

                if (fenceCircle != null) {
                    fenceCircle.setRadius(fenceRadius);
                }

                if (currentLatLng != null && fenceCenter != null) {
                    checkFenceState(currentLatLng, false);
                }
            }

            @Override
            public void onStartTrackingTouch(SeekBar seekBar) {}

            @Override
            public void onStopTrackingTouch(SeekBar seekBar) {}
        });

        btnToggleInfo.setOnClickListener(v -> toggleInfoPanel());

        btnSearch.setOnClickListener(v -> searchLocation());

        lvSearchResults.setOnItemClickListener((parent, view, position, id) ->
        selectSearchResult(position));

        btnWalkRoute.setOnClickListener(v -> planWalkRoute());
    }

```

```

        btnDriveRoute.setOnClickListener(v -> planDriveRoute());
        btnClearRoute.setOnClickListener(v -> clearRoute());

        btnFenceMode.setOnClickListener(v -> toggleFenceMode());
        btnClearFence.setOnClickListener(v -> clearFence());

        btnTakePhoto.setOnClickListener(v -> takePhotoWithLocation());
        btnClearPhotos.setOnClickListener(v -> clearPhotoMarkers());

        btnTrack.setOnClickListener(v -> toggleTrackRecording());
        btnClearTrack.setOnClickListener(v -> clearTrack());

        btnCenterMe.setOnClickListener(v -> {
            if (currentLatLng != null) {
                aMap.animateCamera(CameraUpdateFactory.newLatLngZoom(currentLatLng, 18f));
            } else {
                Toast.makeText(this, "当前位置未获取到", Toast.LENGTH_SHORT).show();
            }
        });
    }

    private void toggleInfoPanel() {
        isInfoExpanded = !isInfoExpanded;
        layoutSearchExtras.setVisibility(isInfoExpanded ? View.VISIBLE : View.GONE);
        btnToggleInfo.setText(isInfoExpanded ? "收起" : "展开");
    }

    private void requestLocationPermissionIfNeeded() {
        if (ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION)
            == PackageManager.PERMISSION_GRANTED) {
            enableLocation();
        } else {
            ActivityCompat.requestPermissions(
                this,
                new String[]{
                    Manifest.permission.ACCESS_FINE_LOCATION,
                    Manifest.permission.ACCESS_COARSE_LOCATION
                },
                REQUEST_LOCATION_PERMISSION
            );
        }
    }

    private void enableLocation() {
        aMap.setLocationSource(this);
        aMap.setMyLocationEnabled(true);
    }

    @Override
    public void activate(OnLocationChangedListener onLocationChangedListener) {
        mListener = onLocationChangedListener;

        if (aMapLocationClient == null) {
            try {
                aMapLocationClient = new AMapLocationClient(getApplicationContext());
            } catch (Exception e) {
                e.printStackTrace();
                return;
            }

            aMapLocationClientOption = new AMapLocationClientOption();
            aMapLocationClient.setLocationListener(this);

            aMapLocationClientOption.setLocationMode(
                AMapLocationClientOption.AMapLocationMode.High_Accuracy
            );
            aMapLocationClientOption.setNeedAddress(true);
            aMapLocationClientOption.setOnceLocation(false);
            aMapLocationClientOption.setInterval(2000);

            aMapLocationClient.setLocationOption(aMapLocationClientOption);
        }

        if (aMapLocationClient != null) {
            aMapLocationClient.startLocation();
        }
    }

```

```

@Override
public void deactivate() {
    mListener = null;

    if (aMapLocationClient != null) {
        aMapLocationClient.stopLocation();
        aMapLocationClient.onDestroy();
        aMapLocationClient = null;
    }
}

@Override
public void onLocationChanged(AMapLocation aMapLocation) {
    if (mListener != null && aMapLocation != null) {
        if (aMapLocation.getErrorCode() == 0) {
            mListener.onLocationChanged(aMapLocation);

            currentLatLng = new LatLng(aMapLocation.getLatitude(), aMapLocation.getLongitude());
            currentAddress = aMapLocation.getAddress() == null ? "未知地址" : aMapLocation.getAddress();
            currentCity = aMapLocation.getCity() == null ? "" : aMapLocation.getCity();

            updateLocationInfo(aMapLocation);

            if (isFirstLocate && currentLatLng != null) {
                aMap.animateCamera(CameraUpdateFactory.newLatLngZoom(currentLatLng, 17f));
                isFirstLocate = false;
            }

            checkFenceState(currentLatLng, true);

            if (isTracking) {
                addTrackPoint(currentLatLng);
            }
        }
    }
}

private void updateLocationInfo(AMapLocation aMapLocation) {
    tvAddress.setText("地址: " + currentAddress);
    tvCoord.setText(String.format(
        Locale.getDefault(),
        "纬度: %.6f 经度: %.6f 精度: %.1fm",
        aMapLocation.getLatitude(),
        aMapLocation.getLongitude(),
        aMapLocation.getAccuracy()
    ));
}

// ===== 围栏 =====

private void toggleFenceMode() {
    isFenceModeEnabled = !isFenceModeEnabled;

    if (isFenceModeEnabled) {
        btnFenceMode.setText("围栏中");
        cardFenceRadius.setVisibility(View.VISIBLE);
        Toast.makeText(this, "围栏模式已开启, 请长按地图设置围栏中心", Toast.LENGTH_SHORT).show();
    } else {
        btnFenceMode.setText("围栏");
        if (fenceCenter == null) {
            cardFenceRadius.setVisibility(View.GONE);
        }
        Toast.makeText(this, "围栏模式已关闭", Toast.LENGTH_SHORT).show();
    }
}

@Override
public void onMapLongClick(LatLng latLng) {
    if (!isFenceModeEnabled) {
        return;
    }

    fenceCenter = latLng;
    lastInsideState = null;
}

```

```

        if (fenceCircle != null) {
            fenceCircle.remove();
        }
        if (fenceMarker != null) {
            fenceMarker.remove();
        }

        fenceMarker = aMap.addMarker(new MarkerOptions()
            .position(fenceCenter)
            .title("电子围栏中心")
            .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_AZURE)));

        fenceCircle = aMap.addCircle(new CircleOptions()
            .center(fenceCenter)
            .radius(fenceRadius)
            .strokeWidth(4)
            .strokeColor(0xAA64748B)
            .fillColor(0x2264748B));

        cardFenceRadius.setVisibility(View.VISIBLE);
        Toast.makeText(this, "电子围栏已设置", Toast.LENGTH_SHORT).show();

        if (currentLatLng != null) {
            checkFenceState(currentLatLng, false);
        }
    }

    private void checkFenceState(LatLng locationLatLng, boolean allowToast) {
        if (fenceCenter == null) {
            updateFenceStatusNoFence();
            return;
        }

        float distance = AMapUtils.calculateLineDistance(locationLatLng, fenceCenter);
        boolean isInside = distance <= fenceRadius;

        String statusText = String.format(
            Locale.getDefault(),
            "围栏: %s | 距中心 %.1fm | 半径 %.0fm",
            isInside ? "围栏内" : "围栏外",
            distance,
            fenceRadius
        );

        tvFenceStatus.setText(statusText);
        tvFenceStatus.setTextColor(isInside ? 0xFF475569 : 0xFF78716C);

        if (lastInsideState == null) {
            lastInsideState = isInside;
            return;
        }

        if (isInside != lastInsideState) {
            if (allowToast) {
                if (isInside) {
                    Toast.makeText(this, "已进入电子围栏", Toast.LENGTH_SHORT).show();
                } else {
                    Toast.makeText(this, "已离开电子围栏", Toast.LENGTH_SHORT).show();
                }
            }
            lastInsideState = isInside;
        }
    }

    private void updateFenceStatusNoFence() {
        tvFenceStatus.setText("围栏: 未设置");
        tvFenceStatus.setTextColor(0xFF64748B);
    }

    private void clearFence() {
        if (fenceCircle != null) {
            fenceCircle.remove();
            fenceCircle = null;
        }
        if (fenceMarker != null) {

```

```

        fenceMarker.remove();
        fenceMarker = null;
    }

    fenceCenter = null;
    lastInsideState = null;
    cardFenceRadius.setVisibility(View.GONE);
    btnFenceMode.setText("围栏");
    isFenceModeEnabled = false;
    updateFenceStatusNoFence();

    Toast.makeText(this, "围栏已清除", Toast.LENGTH_SHORT).show();
}

// ===== 轨迹 =====

private void toggleTrackRecording() {
    isTracking = !isTracking;

    if (isTracking) {
        btnTrack.setText("停轨");
        Toast.makeText(this, "开始记录轨迹", Toast.LENGTH_SHORT).show();
    } else {
        btnTrack.setText("轨迹");
        Toast.makeText(this, "已停止记录轨迹", Toast.LENGTH_SHORT).show();
    }

    updateTrackStatus();
}

private void addTrackPoint(LatLng point) {
    if (trackPoints.isEmpty()) {
        trackPoints.add(point);
    } else {
        LatLng lastPoint = trackPoints.get(trackPoints.size() - 1);
        float distance = AMapUtils.calculateLineDistance(lastPoint, point);

        if (distance >= 2f) {
            trackDistance += distance;
            trackPoints.add(point);
        }
    }

    if (trackPolyline == null) {
        trackPolyline = aMap.addPolyline(new PolylineOptions()
            .addAll(trackPoints)
            .width(10f)
            .color(0xFF64748B));
    } else {
        trackPolyline.setPoints(new ArrayList<>(trackPoints));
    }

    updateTrackStatus();
}

private void clearTrack() {
    isTracking = false;
    btnTrack.setText("轨迹");

    trackPoints.clear();
    trackDistance = 0f;

    if (trackPolyline != null) {
        trackPolyline.remove();
        trackPolyline = null;
    }

    updateTrackStatus();
    Toast.makeText(this, "轨迹已清除", Toast.LENGTH_SHORT).show();
}

private void updateTrackStatus() {
    if (trackPoints.isEmpty()) {
        tvTrackStatus.setText("轨迹: 未记录");
    }
}

```

```

        return;
    }

    if (isTracking) {
        tvTrackStatus.setText(String.format(
            Locale.getDefault(),
            "轨迹: 记录中 | 点数 %d | 距离 %.1fm",
            trackPoints.size(),
            trackDistance
        ));
    } else {
        tvTrackStatus.setText(String.format(
            Locale.getDefault(),
            "轨迹: 已停止 | 点数 %d | 总长 %.1fm",
            trackPoints.size(),
            trackDistance
        ));
    }
}

// ===== 搜索与路线 =====

private void searchLocation() {
    String keyword = etKeyword.getText().toString().trim();
    if (keyword.isEmpty()) {
        Toast.makeText(this, "请输入要搜索的位置", Toast.LENGTH_SHORT).show();
        return;
    }

    isInfoExpanded = true;
    layoutSearchExtras.setVisibility(View.VISIBLE);
    btnToggleInfo.setText("收起");

    PoiSearch.Query query = new PoiSearch.Query(keyword, "", "");
    query.setPageSize(20);
    query.setPageNum(1);

    try {
        PoiSearch poiSearch = new PoiSearch(this, query);
        poiSearch.setOnPoiSearchListener(this);
        poiSearch.searchPOIAsyn();
        tvRouteStatus.setText("路线: 正在搜索位置...");
    } catch (AMapException e) {
        e.printStackTrace();
        tvRouteStatus.setText("路线: 位置搜索初始化失败");
        Toast.makeText(this, "位置搜索初始化失败", Toast.LENGTH_SHORT).show();
    }
}

@Override
public void onPoiSearched(PoiResult pageResult, int errorCode) {
    searchPoiList.clear();

    if (errorCode == 1000 && pageResult != null && pageResult.getPois() != null
        && !pageResult.getPois().isEmpty()) {

        searchPoiList.addAll(pageResult.getPois());
        searchAdapter.notifyDataSetChanged();

        lvSearchResults.setVisibility(View.VISIBLE);
        tvRouteStatus.setText("路线: 请选择一个搜索结果");
    } else {
        lvSearchResults.setVisibility(View.GONE);
        layoutRouteActions.setVisibility(View.GONE);
        tvRouteStatus.setText("路线: 未搜索到结果");
        Toast.makeText(this, "未搜索到结果", Toast.LENGTH_SHORT).show();
    }
}

private void selectSearchResult(int position) {
    if (position < 0 || position >= searchPoiList.size()) {
        return;
    }

    PoiItem item = searchPoiList.get(position);
    LatLonPoint point = item.getLatLonPoint();

```

```

        if (point == null) {
            Toast.makeText(this, "该结果没有有效坐标", Toast.LENGTH_SHORT).show();
            return;
        }

        routeDestination = new LatLng(point.getLatitude(), point.getLongitude());
        routeDestinationName = item.getTitle() == null ? "目的地" : item.getTitle();

        if (destinationMarker != null) {
            destinationMarker.remove();
        }

        destinationMarker = aMap.addMarker(new MarkerOptions()
            .position(routeDestination)
            .title(routeDestinationName)
            .snippet(item.getSnippet())
            .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_ORANGE)));

        aMap.animateCamera(CameraUpdateFactory.newLatLngZoom(routeDestination, 16f));
        lvSearchResults.setVisibility(View.GONE);

        layoutRouteActions.setVisibility(View.VISIBLE);
        tvSelectedDestination.setText("已选目的地: " + routeDestinationName);
        tvRouteStatus.setText("路线: 已选目的地, 未规划");
        Toast.makeText(this, "已选择: " + routeDestinationName, Toast.LENGTH_SHORT).show();
    }

    @Override
    public void onPoiItemSearched(PoiItem poiItem, int errorCode) {
        // 未使用
    }

    private void planWalkRoute() {
        if (currentLatLng == null) {
            Toast.makeText(this, "当前位置未获取到", Toast.LENGTH_SHORT).show();
            return;
        }

        if (routeDestination == null) {
            Toast.makeText(this, "请先搜索并选择一个目的地", Toast.LENGTH_SHORT).show();
            return;
        }

        if (routeSearch == null) {
            Toast.makeText(this, "路线服务初始化失败", Toast.LENGTH_SHORT).show();
            return;
        }

        RouteSearch.FromAndTo fromAndTo = new RouteSearch.FromAndTo(
            new LatLonPoint(currentLatLng.latitude, currentLatLng.longitude),
            new LatLonPoint(routeDestination.latitude, routeDestination.longitude)
        );

        RouteSearch.WalkRouteQuery query = new RouteSearch.WalkRouteQuery(fromAndTo);
        routeSearch.calculateWalkRouteAsync(query);

        tvRouteStatus.setText("路线: 正在规划步行路线...");
    }

    private void planDriveRoute() {
        if (currentLatLng == null) {
            Toast.makeText(this, "当前位置未获取到", Toast.LENGTH_SHORT).show();
            return;
        }

        if (routeDestination == null) {
            Toast.makeText(this, "请先搜索并选择一个目的地", Toast.LENGTH_SHORT).show();
            return;
        }

        if (routeSearch == null) {
            Toast.makeText(this, "路线服务初始化失败", Toast.LENGTH_SHORT).show();
        }
    }

```

```

        return;
    }

    RouteSearch.FromAndTo fromAndTo = new RouteSearch.FromAndTo(
        new LatLonPoint(currentLatLng.latitude, currentLatLng.longitude),
        new LatLonPoint(routeDestination.latitude, routeDestination.longitude)
    );

    RouteSearch.DriveRouteQuery query = new RouteSearch.DriveRouteQuery(
        fromAndTo,
        RouteSearch.DRIVING_SINGLE_DEFAULT,
        null,
        null,
        ""
    );

    routeSearch.calculateDriveRouteAsyn(query);
    tvRouteStatus.setText("路线: 正在规划驾车路线...");
}

@Override
public void onWalkRouteSearched(WalkRouteResult walkRouteResult, int errorCode) {
    if (errorCode == 1000
        && walkRouteResult != null
        && walkRouteResult.getPaths() != null
        && !walkRouteResult.getPaths().isEmpty()) {

        WalkPath walkPath = walkRouteResult.getPaths().get(0);
        List<LatLng> latLngs = new ArrayList<>();

        if (currentLatLng != null) {
            latLngs.add(currentLatLng);
        }

        for (WalkStep step : walkPath.getSteps()) {
            List<LatLonPoint> points = step.getPolyline();
            if (points != null) {
                for (LatLonPoint p : points) {
                    latLngs.add(new LatLng(p.getLatitude(), p.getLongitude()));
                }
            }
        }

        drawRoutePolyline(latLngs, 0xFF1D4ED8);

        tvRouteStatus.setText(String.format(
            Locale.getDefault(),
            "路线: 步行 %.0fm | 约 %.0f 分钟",
            walkPath.getDistance(),
            walkPath.getDuration() / 60f
        ));

        Toast.makeText(this, "步行路线规划成功", Toast.LENGTH_SHORT).show();
    } else {
        tvRouteStatus.setText("路线: 步行路线规划失败");
        Toast.makeText(this, "步行路线规划失败", Toast.LENGTH_SHORT).show();
    }
}

@Override
public void onDriveRouteSearched(DriveRouteResult driveRouteResult, int errorCode) {
    if (errorCode == 1000
        && driveRouteResult != null
        && driveRouteResult.getPaths() != null
        && !driveRouteResult.getPaths().isEmpty()) {

        DrivePath drivePath = driveRouteResult.getPaths().get(0);
        List<LatLng> latLngs = new ArrayList<>();

        if (currentLatLng != null) {
            latLngs.add(currentLatLng);
        }

        for (com.amap.api.services.route.DriveStep step : drivePath.getSteps()) {
            List<LatLonPoint> points = step.getPolyline();
            if (points != null) {

```



```

        for (LatLonPoint p : points) {
            latLngs.add(new LatLng(p.getLatitude(), p.getLongitude()));
        }
    }

    drawRoutePolyline(latLngs, 0xFFFF6B00);

    tvRouteStatus.setText(String.format(
        Locale.getDefault(),
        "路线: 驾车 %.0fm | 约 %.0f 分钟",
        drivePath.getDistance(),
        drivePath.getDuration() / 60f
    ));

    Toast.makeText(this, "驾车路线规划成功", Toast.LENGTH_SHORT).show();
} else {
    tvRouteStatus.setText("路线: 驾车路线规划失败");
    Toast.makeText(this, "驾车路线规划失败", Toast.LENGTH_SHORT).show();
}
}

private void drawRoutePolyline(List<LatLng> latLngs, int color) {
    if (routePolyline != null) {
        routePolyline.remove();
    }

    routePolyline = aMap.addPolyline(new PolylineOptions()
        .addAll(latLngs)
        .width(14f)
        .color(color));

    if (!latLngs.isEmpty()) {
        LatLngBounds.Builder builder = new LatLngBounds.Builder();
        for (LatLng latLng : latLngs) {
            builder.include(latLng);
        }
        aMap.animateCamera(CameraUpdateFactory.newLatLngBounds(builder.build(), 120));
    }
}

@Override
public void onBusRouteSearched(BusRouteResult busRouteResult, int errorCode) {
    // 未使用
}

@Override
public void onRideRouteSearched(RideRouteResult rideRouteResult, int errorCode) {
    // 未使用
}

private void clearRoute() {
    if (routePolyline != null) {
        routePolyline.remove();
        routePolyline = null;
    }
    tvRouteStatus.setText("路线: 已清除");
    Toast.makeText(this, "路线已清除", Toast.LENGTH_SHORT).show();
}

// ===== 拍照 =====

private void takePhotoWithLocation() {
    if (currentLatLng == null) {
        Toast.makeText(this, "当前位置未获取到, 稍后再试", Toast.LENGTH_SHORT).show();
        return;
    }

    if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
        != PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(
            this,
            new String[]{Manifest.permission.CAMERA},
            REQUEST_CAMERA_PERMISSION
        );
        return;
    }
}

```

```

    }

    Intent takePictureIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
    if (takePictureIntent.resolveActivity(getPackageManager()) != null) {
        pendingPhotoLatLng = currentLatLng;
        pendingPhotoTime = timeFormat.format(new Date());
        pendingPhotoAddress = currentAddress;
        startActivityForResult(takePictureIntent, REQUEST_IMAGE_CAPTURE);
    } else {
        Toast.makeText(this, "未找到可用相机应用", Toast.LENGTH_SHORT).show();
    }
}

@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if (requestCode == REQUEST_IMAGE_CAPTURE && resultCode == RESULT_OK && data != null) {
        Bundle extras = data.getExtras();
        if (extras != null && extras.get("data") instanceof Bitmap) {
            Bitmap bitmap = (Bitmap) extras.get("data");

            if (pendingPhotoLatLng != null) {
                Marker marker = aMap.addMarker(new MarkerOptions()
                    .position(pendingPhotoLatLng)
                    .title("拍照定位点")
                    .snippet("点击查看照片")
                    .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_ROSE)
));

                if (marker != null) {
                    String info = "时间: " + pendingPhotoTime
                        + "\n 地址: " + pendingPhotoAddress
                        + "\n 经度: " + pendingPhotoLatLng.longitude
                        + "\n 纬度: " + pendingPhotoLatLng.latitude;

                    photoBitmapMap.put(marker.getId(), bitmap);
                    photoInfoMap.put(marker.getId(), info);
                    photoMarkers.add(marker);

                    aMap.animateCamera(CameraUpdateFactory.newLatLngZoom(pendingPhotoLatLng, 18f));
                    Toast.makeText(this, "拍照定位成功", Toast.LENGTH_SHORT).show();
                }
            }
        }
    }

    private void clearPhotoMarkers() {
        for (Marker marker : photoMarkers) {
            if (marker != null) {
                marker.remove();
            }
        }
        photoMarkers.clear();
        photoBitmapMap.clear();
        photoInfoMap.clear();
        Toast.makeText(this, "拍照点已清除", Toast.LENGTH_SHORT).show();
    }

    @Override
    public boolean onMarkerClick(Marker marker) {
        if (photoBitmapMap.containsKey(marker.getId())) {
            showPhotoDialog(photoBitmapMap.get(marker.getId()), photoInfoMap.get(marker.getId()));
            return true;
        }
        return false;
    }

    private void showPhotoDialog(Bitmap bitmap, String info) {
        LinearLayout layout = new LinearLayout(this);
        layout.setOrientation(LinearLayout.VERTICAL);
        layout.setPadding(dp(20), dp(20), dp(20), dp(10));

        ImageView imageView = new ImageView(this);
        imageView.setImageBitmap(bitmap);

```

```

        imageView.setAdjustViewBounds(true);

        TextView textView = new TextView(this);
        textView.setText(info);
        textView.setTextSize(15f);
        textView.setPadding(0, dp(16), 0, 0);

        layout.addView(imageView);
        layout.addView(textView);

        new AlertDialog.Builder(this)
            .setTitle("拍照定位信息")
            .setView(layout)
            .setPositiveButton("确定", null)
            .show();
    }

    // ===== 生命周期 =====

    @Override
    protected void onResume() {
        super.onResume();
        mapView.onResume();
    }

    @Override
    protected void onPause() {
        super.onPause();
        mapView.onPause();
    }

    @Override
    protected void onSaveInstanceState(@NonNull Bundle outState) {
        super.onSaveInstanceState(outState);
        mapView.onSaveInstanceState(outState);
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        if (aMapLocationClient != null) {
            aMapLocationClient.stopLocation();
            aMapLocationClient.onDestroy();
        }
        mapView.onDestroy();
    }

    // ===== 权限回调 =====

    @Override
    public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions,
                                           @NonNull int[] grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);

        if (requestCode == REQUEST_LOCATION_PERMISSION) {
            boolean granted = true;
            for (int result : grantResults) {
                if (result != PackageManager.PERMISSION_GRANTED) {
                    granted = false;
                    break;
                }
            }
            if (granted) {
                enableLocation();
            } else {
                Toast.makeText(this, "未授予定位权限", Toast.LENGTH_SHORT).show();
            }
        }

        if (requestCode == REQUEST_CAMERA_PERMISSION) {
            if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                takePhotoWithLocation();
            } else {
                Toast.makeText(this, "未授予相机权限", Toast.LENGTH_SHORT).show();
            }
        }
    }

```

```

}

// ===== 工具方法 =====

/**
 * dp 转 px, 方便代码里创建带间距、圆角的搜索结果项
 */
private int dp(int value) {
    return (int) TypedValue.applyDimension(
        TypedValue.COMPLEX_UNIT_DIP,
        value,
        getResources().getDisplayMetrics()
    );
}

// ===== 自定义搜索结果适配器 =====

/**
 * 不新增 XML 文件, 直接在 Java 里动态构建“更美观的搜索结果项”
 */
private class SearchResultAdapter extends BaseAdapter {

    @Override
    public int getCount() {
        return searchPoiList.size();
    }

    @Override
    public Object getItem(int position) {
        return searchPoiList.get(position);
    }

    @Override
    public long getItemId(int position) {
        return position;
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        ViewHolder holder;

        if (convertView == null) {
            // 外层卡片式容器
            LinearLayout root = new LinearLayout(MainActivity.this);
            root.setOrientation(LinearLayout.HORIZONTAL);
            root.setGravity(Gravity.CENTER_VERTICAL);
            root.setMinimumHeight(dp(62));
            root.setPadding(dp(12), dp(10), dp(12), dp(10));

            GradientDrawable bg = new GradientDrawable();
            bg.setColor(0xFF8FAFC);
            bg.setCornerRadius(dp(14));
            bg.setStroke(dp(1), 0xFFE2E8F0);
            root.setBackground(bg);

            // 左侧标题/地址
            LinearLayout textContainer = new LinearLayout(MainActivity.this);
            textContainer.setOrientation(LinearLayout.VERTICAL);
            LinearLayout.LayoutParams textParams =
                new LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f);
            textContainer.setLayoutParams(textParams);

            TextView title = new TextView(MainActivity.this);
            title.setTextColor(0xFF0F172A);
            title.setTextSize(TypedValue.COMPLEX_UNIT_SP, 14);
            title.setTypeface(Typeface.DEFAULT_BOLD);
            title.setMaxLines(1);

            TextView snippet = new TextView(MainActivity.this);
            snippet.setTextColor(0xFF64748B);
            snippet.setTextSize(TypedValue.COMPLEX_UNIT_SP, 12);
            snippet.setMaxLines(2);
            LinearLayout.LayoutParams snippetLp =
                new LinearLayout.LayoutParams(ViewGroup.LayoutParams.WRAP_CONTENT,
ViewGroup.LayoutParams.WRAP_CONTENT);
            snippetLp.topMargin = dp(4);

```

```

        snippet.setLayoutParams(snippetLp);

        textContainer.addView(title);
        textContainer.addView(snippet);

        // 右侧箭头
        TextView arrow = new TextView(MainActivity.this);
        arrow.setText(">");
        arrow.setTextColor(0xFF94A3B8);
        arrow.setTextSize(TypedValue.COMPLEX_UNIT_SP, 18);

        root.addView(textContainer);
        root.addView(arrow);

        holder = new ViewHolder();
        holder.title = title;
        holder.snippet = snippet;
        holder.arrow = arrow;

        convertView = root;
        convertView.setTag(holder);
    } else {
        holder = (ViewHolder) convertView.getTag();
    }

    PoiItem item = searchPoiList.get(position);

    String title = item.getTitle() == null ? "未知地点" : item.getTitle();
    String snippet = item.getSnippet() == null || item.getSnippet().trim().isEmpty()
        ? "暂无详细地址"
        : item.getSnippet();

    holder.title.setText(title);
    holder.snippet.setText(snippet);

    return convertView;
}

class ViewHolder {
    TextView title;
    TextView snippet;
    TextView arrow;
}
}

```

3 结果

3.1 测试环境与测试方法

表 3-1 建议测试环境记录表

项目	填写内容
开发平台	Android Studio（填写你的实际版本）
运行设备	Android 真机 / 模拟器（建议填写型号、系统版本）
地图 SDK	高德地图 Android SDK
网络环境	Wi-Fi / 移动网络
定位方式	高精度定位模式（Hight_Accuracy）
测试场景	室外开阔区域、校园道路或楼宇周边等

3.2 功能运行结果展示

（1）启动与自动定位结果。应用启动后，系统首先显示地图主界面，并自动请求定位权限。定位成功后，顶部状态卡片显示当前地址、经纬度与定位精度，地图镜头自动移动到当前位置。该过程能够证明地图初始化、权限申请、定位回调和状态文本更新链路已经打通。



图 3-1 APP 启动与自动定位界面

(2) 电子围栏设置结果。点击“围栏”按钮进入围栏模式后，用户可长按地图设置围栏中心，程序会在该位置生成一个蓝色围栏中心 Marker 与一个半透明圆形区域，同时弹出“电子围栏已设置”提示。若拖动半径滑条，状态栏中的围栏半径会同步变化，圆形覆盖物也随之实时调整。该现象说明围栏中心设置、圆形渲染和参数联动逻辑均工作正常。

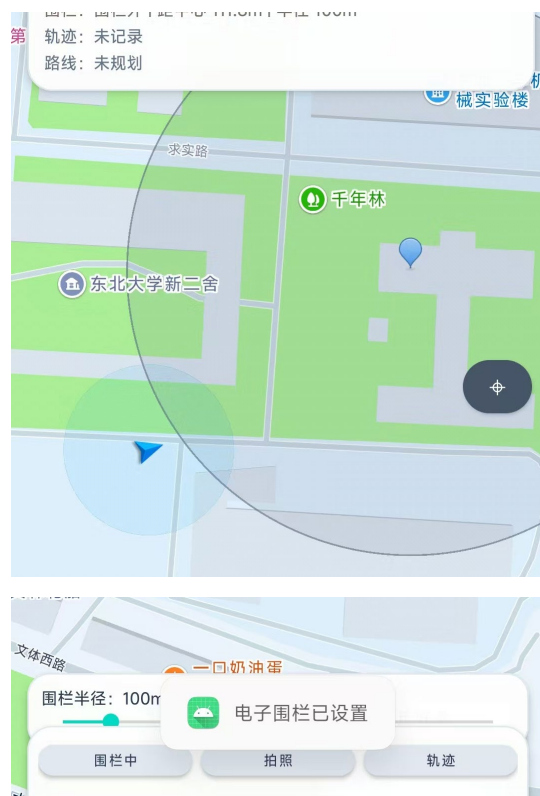


图 3-2 电子围栏设置与半径调节界面

(3) 电子围栏状态变化结果。当用户移动到围栏边界内外时，顶部状态栏会更新为“围栏内/围栏外 | 距中心 xx m | 半径 xx m”，并且只有在状态发生变化时才弹出进入/离开提示。该现象反映出程序已实现基于距离阈值的围栏判定和状态机式的变化检测，避免了连续定位条件下的重复提醒。



图 3-3 进入/离开电子围栏状态展示

(4) 拍照定位结果。点击“拍照”按钮后，系统调用相机拍摄现场图像；拍摄完成返回应用时，地图会在当前位置生成一个玫红色拍照定位点。点击该点后，会弹出包含照片缩略图、拍摄时间、地址及经纬度信息的对话框。该过程说明：相机权限申请、拍照结果回传、照片与位置元数据绑定、Marker 事件响应均已正确实现。

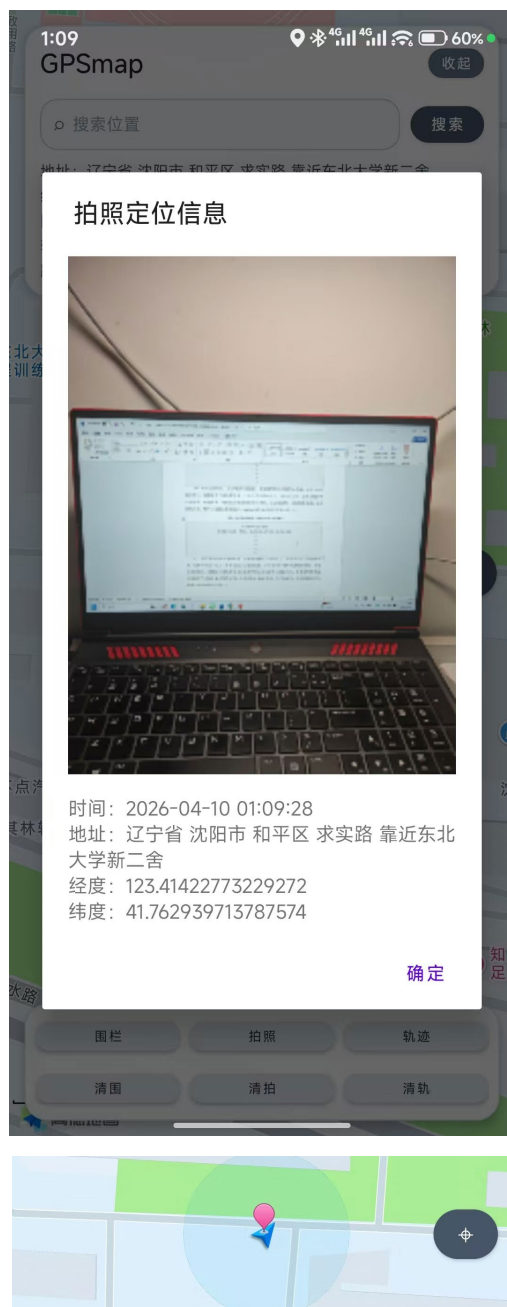


图 3-4 拍照定位点与照片信息对话框

(5) 地点搜索与路径规划结果。在搜索框输入目标地点后，系统会显示一组搜索结果；选择其中某一项后，界面会显示已选目的地，并开放步行/驾车路线规划按钮。若选择步行模式，地图绘制蓝色折线并在状态栏显示步行距离与预估时间；若选择驾车模式，则绘制橙色折线并显示驾车结果。由此可验证 POI 搜索、目的地解析、路径回调解析与多模式路线绘制均有效。



图 3-5 搜索结果、目的地选择与路线规划界面

(6) 轨迹记录结果。点击“轨迹”按钮后，程序开始持续记录用户运动轨迹；当用户移动距离超过 2 m 时，新轨迹点被加入列表并绘制到地图上，状态栏同步显示点数与累计距离。停止记录后，状态信息变为“已停止”，而轨迹线保留在地图上，便于回看。该过程表明轨迹采样阈值、距离累加与折线更新逻辑实现正确。



图 3-6 轨迹记录与轨迹清除界面

3.3 结果分析

从功能完整性角度看，当前 APP 已经形成了“定位—搜索—路线—围栏—拍照—轨迹”六个环节相互衔接的业务链条，明显超出了仅实现两项新增功能的最低要求。电子围栏和拍照定位是关键的新增模块：前者把空间范围监测能力引入了移动端地图应用，后者把位置与现场图像信息绑定起来，增强了定位结果的可记录性和可解释性。

从算法实现角度看，程序并未堆砌复杂模型，而是针对移动端实时应用场景选择了轻量、直接且稳定的实现方式。例如围栏判定直接建立在地图 SDK 提供的测距函数之上，既保证了距离计算的可靠性，又简化了工程复杂度；轨迹记录则利用最小距离阈值抑制静止漂移和重复采样，使累计轨迹长度更加平稳；拍照定位则通过 Marker ID 与 HashMap 建立照片索引，数据组织清楚，事件回调简单有效。

从界面交互角度看，当前布局相对紧凑且适合手机端演示。顶部搜索与状态卡片承担“信息查看”职责，底部按钮区承担“功能操作”职责，右侧定位按钮承担“快速回到当前位置”职责，围栏半径单独放置在小卡片中，避免与主按钮区混杂。这样的界面分层使用户能够在不离开地图主视图的前提下完成大部分操作，比较符合导航类 APP 的使用习惯。

不过，结果分析也必须指出当前原型的局限性。第一，拍照结果目前只保存了相机返回的缩略图，没有保存原始图像和历史记录，因此无法长期管理拍照点；第二，电子围栏依赖前台持续定位，尚未扩展为系统级后台围栏监控或通知机制；第三，搜索与路线规划依赖在线地图服务，网络不佳时可能影响使用体验；第四，虽然题目示例中提到 CORS/PPP 高精度定位，但当前工程尚未接入外部高精度改正服务，因此定位精度仍主要由移动端地图 SDK 输出决定。换言之，当前成果更准确地说是一个“以电子围栏与拍照定位为核心的导航定位 APP 功能增强原型”。

若后续继续扩展，本项目可沿着三个方向深化：一是做“数据持久化”，把拍照点、轨迹与围栏配置保存到本地数据库；二是做“高精度定位增强”，引入 GNSS 原始观测、RTK/PPP 或网络差分改正，提高定位精度；三是做“后台服务化”，把围栏越界提醒、轨迹记录和照片管理封装为长期运行的服务模块。这样即可从课程设计原型进一步提升到更接近实际应用的工程版本。

4 总结

本文按照任务二的要求，在已有 Android 导航定位 APP 代码基础上完成了系统化改进，并围绕“电子围栏”和“拍照定位”两项新功能进行了重点实现与说明。结合上传代码可以看到，程序已经具备实时定位、地址显示、地点搜索、步行/驾车路线规划、电子围栏设置与状态监测、拍照定位标记、轨迹记录以及界面交互优化等多项能力，整体功能链条较完整，能够满足课程大作业对“导航定位手机 APP 改进”的展示需求。

从课程学习角度看，本次作业的价值不仅在于把若干 API 调用拼接到一起，更在于理解移动定位应用中的“位置状态—地图对象—用户事件—业务逻辑”之间是如何相互耦合的。电子围栏体现了空间距离判定与状态变化检测的思路，拍照定位体现了多源信息绑定与地图标记管理的思路，轨迹记录体现了移动数据采样与噪声抑制的思路，而搜索与路线规划则体现了基于地图服务的目标点解析与路径可视化过程。通过这些模块的联合实现，可以较好地把课堂上关于导航定位应用开发的知识落实到一个可运行的移动端原型中。

当然，当前系统仍然是一个面向课程作业的实现版本，在数据持久化、后台任务、高精度 GNSS 增强和异常场景处理等方面还有继续完善的空间。但从现阶段来看，该项目已经完成了题目要求的核心目标：在已有代码基础上实现至少两项与导航定位密切相关的新功能，并形成了结构清晰、逻辑完整、便于展示与答辩的 APP 工程。后续若结合更高精度的 GNSS 解算模块或差分改正服务，还可以进一步向专业测绘导航应用方向拓展。